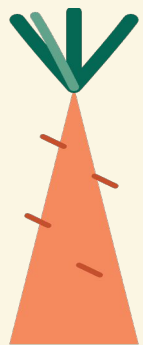


APAN 5310 · Group 2 · Spring 2026

ABC Foodmart

–Database System–

A consultant engagement to transform 30+ years of paper records and spreadsheets into a scalable relational database.

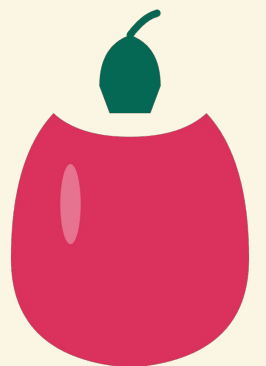
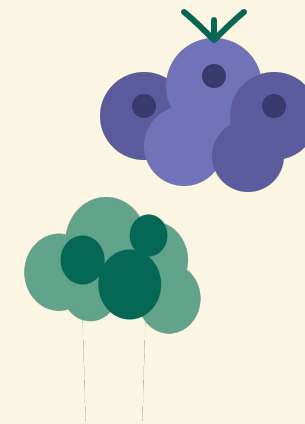


Yike Meng
ym3137

Maggie Teng
mt3952

Olivia Chou
cc5399

Afeefah Moazzam
am6976



Meet Our Client: ABC Foodmart



ABC Foodmart

Neighborhood grocery · Queens, NY

30+

Years in operation

2 → 5

Stores in expansion

- All data managed through spreadsheets & paper binders
- New management is expanding to Brooklyn, but legacy systems can't scale

PROBLEM

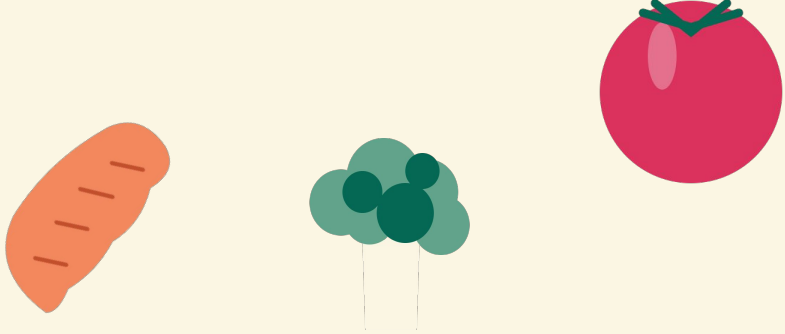
- ✗ No centralized inventory view across stores
- ✗ Sales & revenue siloed — manual cross-store comparison
- ✗ Vendor & delivery records on paper — risk of loss
- ✗ Payroll calculated by hand — disconnected from schedules
- ✗ Zero customer / loyalty tracking

TASK

Design a centralized database that **replaces spreadsheets**, automates inventory & payroll, and lets both store managers and the C-suite see what's happening right now.



Original Data Populated



5 Stores

2 legacy locations in **Queens**
3 new locations in **Brooklyn**

20 Departments

4 departments per store
50 employees · 10 per store

1K Transactions

200 per store · 3,726 line items
200 customers

8 Vendors

COMPANY
FreshFarms Wholesale
Metro Dairy Supply
BakePro Distributors
Green Valley Produce
National Frozen Foods
Sunrise Beverage Co
QuickSnack Wholesale
Harbor Seafood Supply

50 Products · 5 Categories

Produce

Bananas \$0.99
Apples \$3.49

Dairy & Frozen

Whole Milk \$3.99
Greek Yogurt \$5.99

Bakery

White Bread \$2.99
Bagels \$3.99

Beverages

Coca-Cola \$6.99
Coffee \$8.99

Snacks

Potato Chips \$3.49 Mixed
Nuts \$7.99



Schedules

600 rows · ~12 / employee

Deliveries

~6/store · 3-6 items/delivery

Promotions

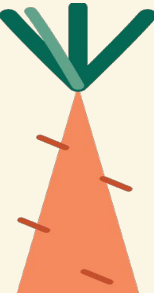
20 rows

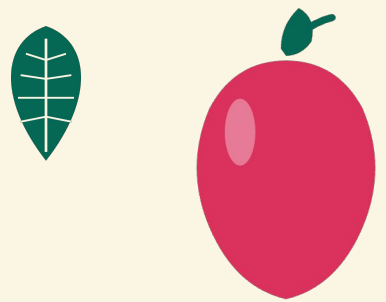
Expenses

150 rows · 10/store/month

Payroll

150 rows · 1/emp/month





16 tables, 4 modules, fully 3NF

Store & Staff

Stores

store_id (PK)
store_name, borough
manager_id → Emp

Departments

dept_id (PK)
store_id → Store

Employees

emp_id (PK)
role, hourly_wage
store_id, dept_id

Schedules

schedule_id (PK)
emp_id → Emp

Supply Chain

Vendors

vendor_id (PK)
company_name

Products

product_id (PK)
category, price
vendor_id

Inventory

inventory_id (PK)
product_id, store_id
qty_on_hand

Deliveries

delivery_id (PK)
vendor_id, store_id

Delivery_Items

item_id (PK)
delivery_id, product_id

Sales & Customers

Customers

customer_id (PK)
name, loyalty_pts

Transactions

trans_id (PK)
customer_id, store_id
total, ts

Transaction_Items

ti_id (PK)
trans_id, product_id
qty, line_total

Promotions

promo_id (PK)
product_id
discount, dates

Finance

Expense_Categories

cat_id (PK)
cat_name

Expenses

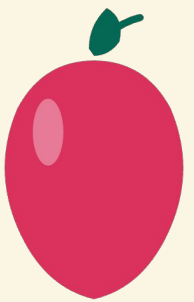
expense_id (PK)
cat_id, store_id
amount, date

Payroll

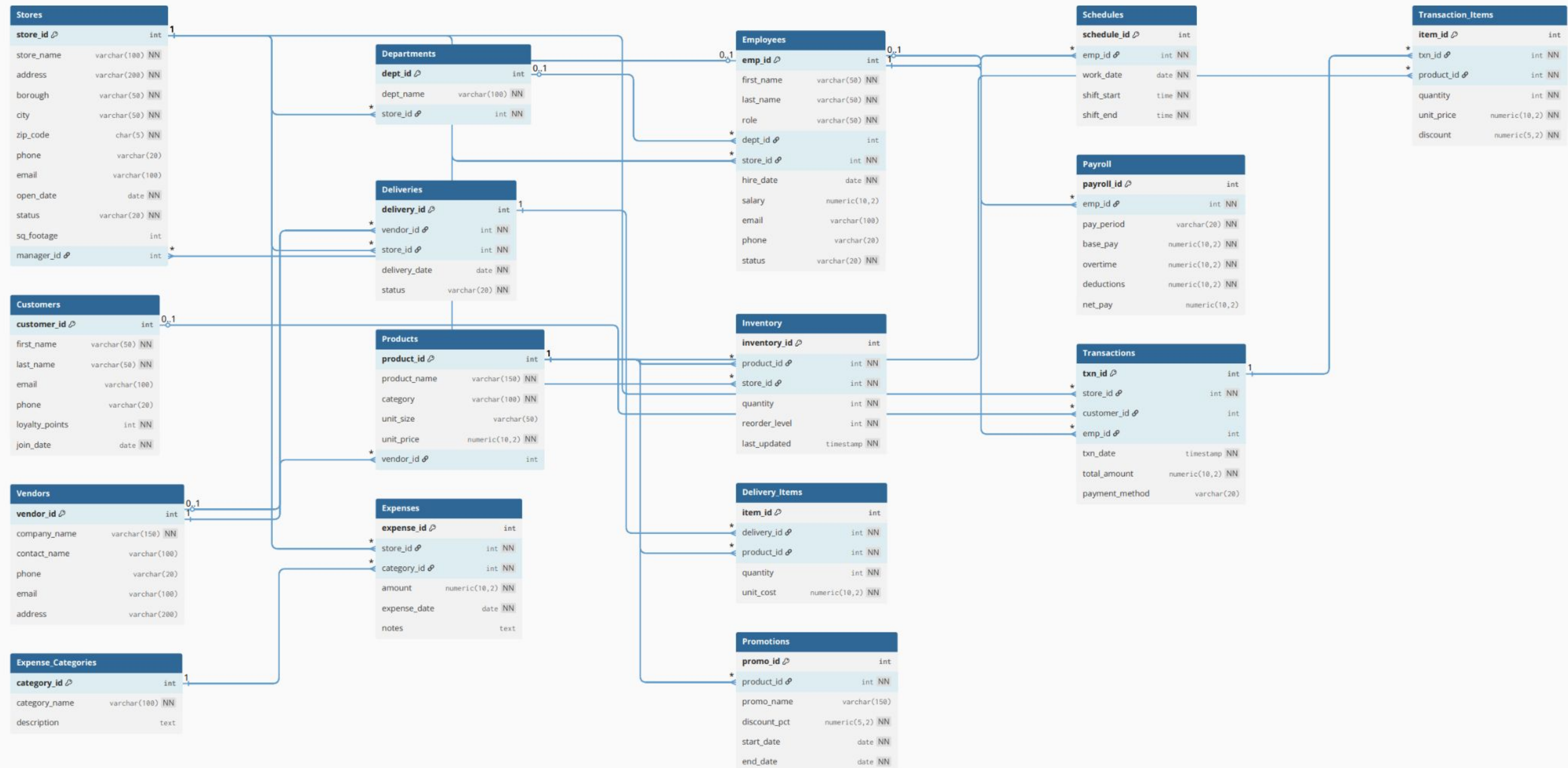
payroll_id (PK)
emp_id
net_pay (computed)

- ✓ Circular FK via **ALTER TABLE** ✓
- net_pay** = GENERATED ALWAYS AS
- ✓ 3 triggers: inventory, loyalty, ts



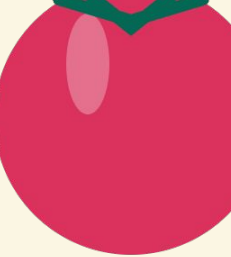


ERD Diagram





From spreadsheet → relational, in 6 steps

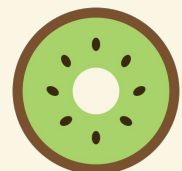


Insertion order:

- 1 Stores**
no manager FK yet
- 2 Depts & Employees**
10 per store
- 3 UPDATE manager_id**
resolves circular FK ⚡
- 4 Vendors → Inventory**
cost = 50-70% of price
- 5 Customers → Trans.**
stock check pre-insert
- 6 Expenses + Payroll**
wage x hours

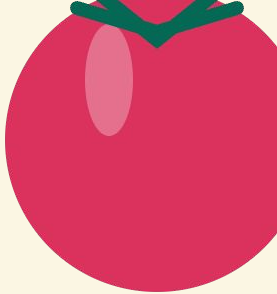
Volumes Loaded

- | | |
|------------------------------|------------------------------|
| 5
Stores | 50
Employees |
| 200
Customers | 600
Schedules |
| 1,000
Transactions | 3,726
Trans. items |
- ✓ **6,300+ records**
across all 16 tables, single transaction



10 Analytical Queries

Questions the C-suite Actually Asks



1 Revenue by Store

Which locations earn the most? Bench Brooklyn vs Queens.

FINANCE

2 Product Revenue Rank

Rank every SKU by revenue — filter by threshold, not fixed N.

SALES

3 Low-Stock Alerts

SKUs at or below reorder level — sorted by urgency per store.

OPERATIONS

4 Labor Cost by Store/Mon

Payroll trends — find stores with disproportionate labor spend.

FINANCE

5 Top Loyalty Customers

Spend × visits × points — who to target with retention offers.

CUSTOMER

6 Daily Revenue per Store

Day-by-day patterns to time staffing & restocking.

SALES

7 Vendor Reliability

% deliveries completed — renegotiate or switch suppliers.

OPERATIONS

8 Overtime by Employee

Who's hours burning? HAVING-clause flags chronic overtime.

HR

9 Category Share per Store

% of revenue per category — tailor inventory to each borough.

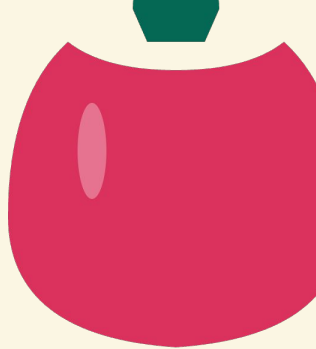
FINANCE

10 Month-Over-Month Growth

LAG() compares each month to prior — clear growth trajectory.

FINANCE





From **paper pain** → **decisions in seconds**

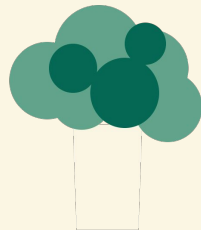
WHAT THEY NEEDED	WHAT WE BUILT	INSIGHT THEY GET
 No live view of stock across stores <i>“Did Brooklyn run out of milk again?”</i>	Inventory + Deliveries module Real-time qty per store · reorder thresholds · auto-flag.	Reorder before shelves go empty ~75 SKUs flagged daily · zero stockouts on top 10 items
 Sales siloed per store, hand-compared <i>“Which store is actually winning?”</i>	Transactions + Products + Promotions Every basket linked to store · category · promo · employee · customer.	Borough & category benchmarking Brooklyn growing +18–22% MoM · confirms expansion
 Payroll calculated by hand <i>“Why is overtime eating the margin?”</i>	Schedules + Payroll module Hours × wage rates → automatic period totals · OT flagged.	Right-size staffing per shift Catch chronic OT before it hits the P&L
 Zero customer / loyalty data <i>“Who are our regulars?”</i>	Customers + loyalty points Spend × visits × baskets, tied back through Transactions.	Targeted offers to top 20% Retain neighborhood loyalty as Brooklyn ramps

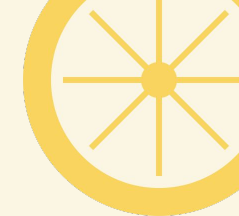


One source
No more reconciling spreadsheets across 5 stores.

Decisions in seconds
Questions that took days now resolve via dashboard click.

Built to scale
2 stores → 5 → 50:
same schema, same dashboards.





Two Access Tiers — Built to Fit Each User's Skill



For Analysts

Direct SQL + Python

```
-- Q1: Revenue by store
SELECT s.store_name, SUM(ti.qty * p.unit_price) AS
revenue FROM Stores s JOIN Transactions t USING (store_id) JOIN
Transaction_Items ti USING (txn_id) JOIN Products p USING (product_id) GROUP
BY s.store_name ORDER BY revenue DESC;
```

SQL

Faker

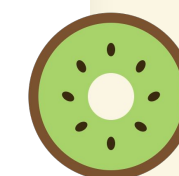
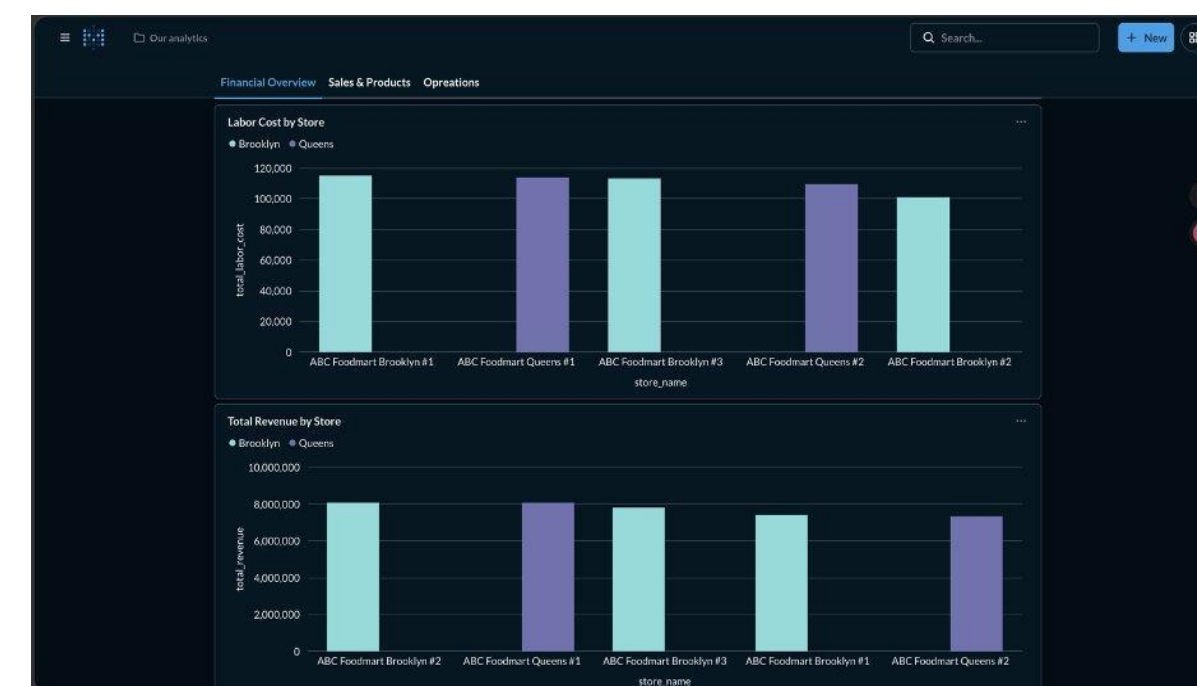
psycopg2

pandas



For Managers & C-Suite

Metabase dashboards



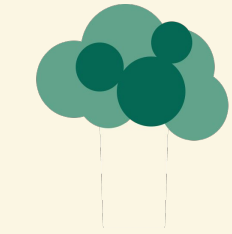
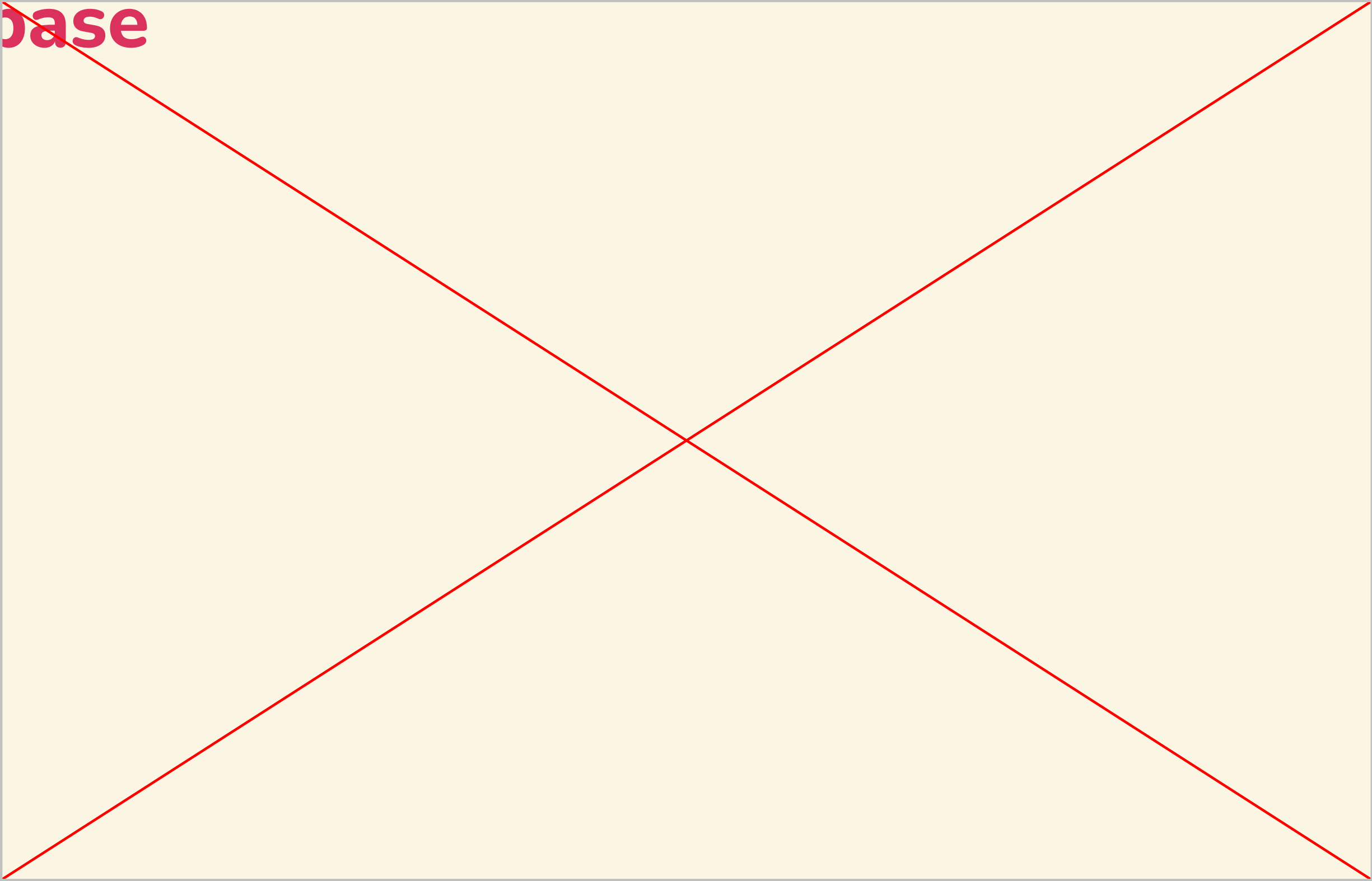
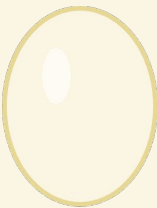
Data Analyst
Raw SQL · Python

Store Manager
Metabase · filtered by store

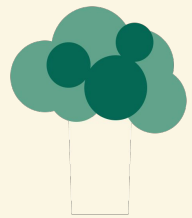
C-Level Executive
Live KPIs · no SQL needed



Live: SQL + Metabase



Results & Impact



16

Tables in 3NF

3

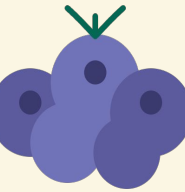
Auto Triggers

10

SQL Queries

6.3K

Sample Records



Real-time visibility — live cross-store data

Scalable — new stores, zero schema changes

Data integrity — FK + triggers, no manual errors

Exec access — Metabase, no SQL needed

